Executable Archaeology of Early Autonomous Programming Systems: Reanimating the 1961 Heuristic Compiler and the Programmer's Apprentice's 1984 KBEmacs

Jeff Shrager[1], Kent M. Pitman[2], and Mia Golec[3]

[1]https://jshrager.org
[2]https://github.com/netsettler
[3]Carnegie Mellon University, Pittsburgh, PA, USA

Abstract

Executable Archaeology is the practice of studying historical computational systems by running their original code in a process called "reanimation". Although AI-assisted programming is usually considered a new invention, the idea of a program that helps write programs is nearly as old as artificial intelligence itself. Here we report on the reanimation of two early programs concerned with programming, but developed for quite different purposes. The first, Herbert Simon's Heuristic Compiler (1961), was an experimental model of programming as problem-solving, applying the means-end analysis methods of the General Problem Solver to investigate the nature of the programming task. The second, KBEmacs (1984), was a working prototype of an intelligent programming assistant developed by MIT's Programmer's Apprentice project. It let a programmer build up programs by combining standard "clichés" through a stylized English command language. Our reanimation of the Heuristic Compiler reproduces the compiled code printed in Simon's 1963 paper, and our reanimation of KBEmacs reproduces the SQRT sequence of its December 1984 self-test. In order to highlight the reflexivity of this project, we tried to have most of the detailed effort in these reanimations carried out by a modern AI-assisted programming tool, Anthropic's Claude Code, although it started from a significant base of existing software, including a modern IPL-V interpreter, modern Common Lisp, and an existing Common Lisp port of the Lisp Machine's object system. Beyond their engineering, both systems embodied theories of how people program—Simon's, of programming as means-end problem solving; the Programmer's Apprentice's, of programming as the recognition and combination of familiar clichés—a dimension largely lost in today's far more capable AI coding assistants.

Keywords: Executable Archaeology, History of AI, Automatic Programming, Heuristic Compiler, KBEmacs, Programmer's Apprentice

1 Introduction

Modern programmers increasingly work alongside AI-based tools built on large language models and trained on enormous corpora of source code. Although the scale and fluency of the modern tools are truly novel, the aspiration of a program that takes part in the work of programming is nearly as old as programming itself. In the 1950s what we now call compilation was referred to as "automatic programming" [1]. And within a few years of the founding of artificial intelligence, researchers were asking whether the methods of AI could be turned on the task of programming itself [2, 17].

This paper describes the reanimation of two historical systems representing different ways in which artificial intelligence was applied to programming: one as an object of scientific investigation, the other as a task to be assisted. The first is Herbert A. Simon's *Heuristic Compiler*, written in IPL-V in 1961 and described in a RAND report that year [23] and in the *Journal of the ACM* in 1963 [25]. Simon asked "whether the task of writing a computer program can be regarded as a 'problem' in the sense in which that term is used in GPS," the General Problem Solver of Newell, Shaw, and Simon [11]. Given a description of what a routine should do, Simon's program searches for differences between the routine as it stands and the desired routine, and applies operators to reduce those differences.

The second system is *KBEmacs*, the Knowledge-Based Editor (in Emacs), built at MIT's Artificial Intelligence Laboratory as the culminating demonstration of the Programmer's Apprentice project [29, 31, 16]. The user of KBEmacs chose which standard algorithms ("clichés") to combine, and directed KBEmacs in stylized English. KBEmacs supplied the details and wrote the code.

These two systems are separated by a quarter of a century, by programming language (IPL-V on 1960s mainframes versus Zetalisp on MIT Lisp Machines and their descendants), and by research tradition, yet they stake out most of the conceptual territory that today's AI coding tools occupy: How a programming task is specified, where the knowledge needed to carry it out comes from, how work is divided between human and machine, and how the system checks and explains what it has done.

We approached both as exercises in "reanimation", the primary methodology underlying *Executable Archaeology*: studying historical computational systems by running them on emulation stacks, from code that is as close as possible to the original [8, 21, 10]. The Heuristic Compiler was transcribed from a 1961 line-printer listing in Simon's papers and run on the Common Lisp IPL-V interpreter built earlier to run the Logic Theorist. KBEmacs was transcribed from a four-volume January 1985 printout and run under a new Zetalisp compatibility layer for SBCL, a modern Common Lisp. Both reanimations were themselves carried out largely by an AI coding assistant: Anthropic's Claude [4], working in its command-line form, Claude Code. The division of labor is described in "Author Contributions and Use of AI".

2 Historical Background

2.1 The Heuristic Compiler

The phrase "automatic programming" has meant different things in different decades, usually whatever was just beyond the current state of the art. For the designers of FORTRAN it meant translating algebraic formulas into machine code [1]. Such translators are deterministic, and were neither then, nor are they now, considered to be AI. Newell, Shaw, and Simon's work at RAND and Carnegie Tech in the 1950s took a different view of what computers could do. Their Logic Theorist (1956) searched for proofs of theorems in *Principia Mathematica* [12, 10], and their General Problem Solver (1959) generalized these methods into means-end analysis [11]. Simon's 1963 summary of the method is worth quoting in full, because the Heuristic Compiler implements it almost literally:

> Given the present and desired objects, find a difference between them. Next, find an operator relevant to the difference; determine if the operator can be applied to the present object. If so, apply it. (If not, describe the objects to which it would apply and transform the present object into an object of that kind—a new "desired object.") Take the new object thus obtained as the present object and repeat the process. [25, p. 494]

A proof and a program are not as different as it might seem; a proof is a sequence of operators that transforms axioms into a theorem. In other words, a proof is a program that generates a theorem as output from the inputs that are axioms. In this sense (Simon continues), "a theorem-proving system can be regarded, at least formally, as a program-writing system. Conversely, if we can formulate a programming goal as a difference between a present and a desired object, we can presumably use the same processes ... to generate a program" [25, p. 495]. The same observation, in a deductive rather than heuristic setting, later became the foundation of program synthesis from theorem proving [6, 9].

Simon described the Heuristic Compiler in various publications and reports in the early 1960s [23, 25, 26]. He was explicit that his goal was *not* to build an automatic programming tool: "no claims will be made for the heuristic programming procedures described here as practical approaches to the construction of compilers. Their interest lies in what they teach us about the nature of the programming task" [25, p. 493].

2.2 The Programmer's Apprentice

The Programmer's Apprentice (PA) project began at MIT in the mid-1970s, with Charles Rich and Howard Shrobe's *Initial Report on a LISP Programmer's Apprentice* [14]. These researchers were realistic about the potential capabilities of such an AI-based programming tool at that time: "Recognizing that it will be a long time before it is possible to fully duplicate an expert programmer's abilities, the project seeks to develop an intelligent assistant system ... which will help a programmer in various phases of the programming task" [31, p. 47].

Over the following decade the project developed three central ideas: *Clichés* are standard computational methods, such as a linear search, a report generator, or successive approximation, each with *roles* that vary from use to use and a fixed surrounding structure. *Plans* are a language-independent representation of programs as hierarchical flowcharts with explicit data and control flow, which Rich formalized in a Plan Calculus [15, 16]. And the *assistant approach* itself was a division of labor wherein the programmer makes design decisions and the apprentice handles the routine details. Waters's dissertation developed the analysis of programs into plans [28].

KBEmacs was the realization of these ideas in a working editor. It ran on MIT Lisp Machines, and their descendants, inside the Zmacs editor, and handled both Lisp and Ada. Its best-known demonstration builds an Ada report generator, `UNIT-REPAIR-REPORT`, from five commands such as "Define a simple-report procedure UNIT-REPAIR-REPORT" and "Fill the enumerator with a chain-enumeration of UNITS and REPAIRS" [31, p. 48],[30].

KBEmacs was admittedly a research prototype. Waters wrote that it knew "only a few dozen clichés" where a practical tool "would need to know many hundreds," and that, its ideas having been used "for essentially all [they are] worth," KBEmacs "has been mothballed" [31, pp. 54–55]. The project continued with work on the Plan Calculus, on automated reasoning, and on requirements and design apprentices, culminating in Rich and Waters's 1990 book [18].

3 Reanimating the Heuristic Compiler

3.1 The Listing

The source used here for the Heuristic Compiler is a line-printer listing headed `HEURISTIC CODER 7/16/61`, discovered in the Herbert A. Simon Papers at the Carnegie Mellon University Archives [24]. This listing is dated shortly after RAND report P-2349, and is very nearly the program described in that report. The listing is 94 line-printer pages, and about 4,470 lines of code

(cards). We call this "the deck". The listing also carries various hand markings indicating a handful of proposed corrections. The deck contains an executive and two compilers; a natural-language front end (a character scanner, dictionary, and parser) intended to let the compiler accept English definitions; and a small knowledge base about IPL-V, encoded as data, which the compilers use to reason. The listing ends with the data and has no start card (IPL-V type 5 card).[1]

3.2 Everything Is a Description List

The central representational idea of the Heuristic Compiler[2] is that a *routine* is an object with attributes stored as an IPL-V description list (approximately a lisp property list). Each attribute is one possible definition of the routine: its IPL name, its *functional description* (i.e., a process name plus arguments, such as "INSERT ( ) AT END OF ( )"), its *state description* (i.e., which cells it affects, and what each holds before and after), its flow diagram, and its IPL-V code. Within the IPL code rendered in this representation, IPL-V instructions are themselves description lists with attributes for each field. And a symbol such as `J3` is a description list with a region (the letter J) and a location (the integer 3). The program never handles IPL code as text. Its compilers build and transform these structures, and separate print routines render them as IPL-V text only at the end.

The Heuristic Compiler's executive implements a means-end problem-solving loop whose *differences* are kinds of definitions and whose *operators* are whole compilers (described just below). The executive finds the closest definition for the routine, looks up the relevant compiler in a table of connections, applies it, and tests whether the routine has improved. If the routine has not improved, the executive gives up.

The *state description compiler* is the most GPS-like part of the Heuristic Compiler. It compares the input and output states of each affected cell, classifies the largest difference (addition, deletion, replacement, or copying), looks up an operator relevant to it, binds the operator's variables to the actual cells, and then recursively compiles two subproblems: getting from the original input state to the required state, and getting from the operator's output state to the desired output. Its knowledge of IPL-V consists of exactly two operators, LOAD and REPLACE, each itself a tiny routine with its own state description.[3]

The *functional description compiler* works by analogy. Given a routine described as, say, "INSERT (1) AT END OF THE VALUE OF THE ATTRIBUTE (0) OF (2)," it searches the routines already compiled for one with the same *process*, here "INSERT (0) AT END OF (1)," whose code is the single primitive `J65`. It then adapts that routine to the new arguments, moving the inputs into working cells, compiling code to fetch each argument, recursing when an argument is itself a described computation (a "determiner"), and wrapping the result in instructions that save and restore the working cells.

3.3 Running The Heuristic Compiler

The deck runs on the Common Lisp IPL-V interpreter originally written to run the Logic Theorist [21, 22]. Claude extended the interpreter with several primitives (J-functions) needed by the

[1] We borrowed the start card from [26].

[2] Simon uses "compiler" at two levels. The Heuristic Compiler as a whole is a compiler; within it, two "compilers," one for state descriptions and one for functional descriptions, serve as the operators of its means-end loop. The final step, in which the code they build as list structures is printed as IPL-V text, is done by ordinary print routines, what we would now call a renderer.

[3] The state description compiler in [26] differs from the one we reanimated in representing states by bare symbols, whereas ours uses described states.

Heuristic Compiler that were not required by the Logic Theorist. It also added a start card. With these changes, the program runs and reproduces Simon's published examples.

Compiling from a functional description. Given the functional description of INSERT AT END OF VALUE LIST, the program produces `J13 J52 11W2 11W0 J10 11W1 J65 J32 0`, character for character the code in the 1963 paper [25, pp. 499–500] (Figure 1). A print left in routine U113 shows the code growing one argument at a time, which reproduces the sequence of stages through which the paper walks its readers. In Simon's runs, as in ours, the functional description is supplied as a hand-coded list structure, not as typed English; the phrase is printed from it, but the compiler matches processes by symbol identity. The English front end in the deck has not yet been run (see "Next Steps").

```
FIND THE VALUE-OF THE ATTRIBUTE (0) OF (1)              <- an already-compiled routine

                                    J10   0

                                    J65   0             <- the code of INSERT (0) AT END OF (1)

                                  11W1                  <- fetch argument (1), now in W1
                                    J65   0

                                  11W2                  <- fetch the determiner's arguments
                                  11W0                     and apply its code (J10)
                                    J10
                                  11W1
                                    J65   0

J13
INSERT (1) AT END OF  THE VALUE-OF THE ATTRIBUTE (0) OF (2)

                             J13    J52                 <- preserve working cells W0-W2
                                  11W2
                                  11W0
                                    J10
                                  11W1
                                    J65
                                    J32   0             <- restore them and exit
```

Figure 1: The 1961 Heuristic Compiler, running in 2026, compiling INSERT AT END OF VALUE LIST from its functional description. The intermediate stages are printed by the program itself (via a print in routine U113 that appears to be a debugging aid left in the deck). The final code is character for character the code printed in Simon's 1963 paper. Annotations after `<-` are ours.

Compiling from a state description. Simon's running example of the state description compiler is J3, SET SIGNAL MINUS: a routine that replaces the top symbol of the signal cell H5 with the symbol J3 ("minus"). The state description says that H5 goes from *some symbol on top of the rest of its pushdown list* to *J3 on top of the same rest*. The compiler finds a replacement difference in a named cell; finds REPLACE relevant, with its variable bound to H5, giving `20H5`; sets up the prefix subproblem "get J3 into the communication cell," an addition, for which LOAD is relevant, giving `10J3`; finds no suffix subproblem; and assembles `10J3 20H5 0`, the code in the paper [25, p. 498].

Surprisingly, as listed, the deck does not compile J3. The listing's J3 has both a functional and a state description, and the executive always prefers the functional description, on the assumption

that "it is easier to compile from a functional description than from a state description" [25, p. 502]. As no already-compiled routine shares J3's process, the functional description compiler fails, the executive sees no progress, and it gives up without ever trying the state description. The paper's J3 result is obtained only when J3's functional description is removed before the run, after which the state description compiler produces exactly the published code. In the 1963 listing (in RM-3588-PR) its executive is identical, but the cards that give J3 a functional description are numbered as later insertions, suggesting that the paper's J3 came from a version in which J3 had only a state description.

Flow diagrams. The paper's last section handles routines with branches and loops by decomposing them into *segments* joined by a flow diagram. The program reassembles routine J77 ("test whether there is a symbol equal to 0ACCUMULATOR on list 1ACCUMULATOR") from its segments, and decomposes it again into the paper's flow diagram.

The information annexer. RM-3588-PR's Appendix B, a small program that stores statements such as "X114 is the X33 of the X25 of X105" into description lists, either identifying an object already in memory or creating one, was also transcribed and runs. Its output matches the fragment of 1963 output that survives in the memorandum [26, 19].

4 Reanimating KBEmacs

4.1 The Listings

The surviving KBEmacs source is a January 1985 printout, four bound volumes of Zetalisp listings, preserved by Kent Pitman, a member of the KBEmacs team, whose scans are at `https://github.com/netsettler/netsettlement/tree/main/kbe` [13]. These cover the system as of KBE version 358.0x (December 1984), although individual files carry their own dates and versions. Volume 1 opens with something unusual and extremely useful to an archaeologist: the results of KBEmacs's automatic self-test, run "Friday the fourteenth of December, 1984; 2:25:23 pm" on the Lisp Machine *Merlin*. The self-test runs six command sequences, compares the code KBEmacs generates with expected code, and reports that three sequences (HASH, SQRT, and SUMLIST) succeeded and three (BUCKET, DELETE, and MEMQ) failed with "Bad code generated."

The 87 files were transcribed (mostly by ChatGPT [3]) into about 34,500 lines of source, now at `https://github.com/jeffshrager/kbemacs` [20], roughly consistent with Waters's description of the system as "some 40,000 lines of Lisp code" [31, p. 55].

4.2 Text and Plans

A KBEmacs user works in a stylized English command language phrased in terms of clichés and their roles. The cliché library is written as code with holes, as in the definition of successive approximation in Figure 2.

```
(DEFINE-CLICHE SUCCESSIVE-APPROXIMATION
   (PRIMARY-ROLES (INITIAL-APPROXIMATION)
    DESCRIBED-ROLES (INITIAL-APPROXIMATION CRITERION IMPROVEMENT-STEP)
    COMMENT "iteratively improves an approximation until {the criterion} is
             satisfied" ...)
  (LET* ((APPROXIMATION {the input initial-approximation}) )
    (LOOP DO
      (IF ({the criterion} APPROXIMATION) (RETURN APPROXIMATION) )
      (SETQ APPROXIMATION ({the improvement-step} APPROXIMATION) ))))
```

Figure 2: A cliché from KBEmacs's library (`kbelib.lisp`, 1984). The braces are KBEmacs read syntax marking *roles*, the parts that vary from use to use; the surrounding code is fixed.

Clichés are not stored as text. When the library loads, each definition is run through KBEmacs's *analyzer*, which turns code into a plan: a set of facts in a small database describing segments, ports, data flow, control flow, and roles. In a plan, a `LOOP` with a `SETQ` disappears; what remains is a temporal composition of a sequence of states. The knowledge-based editor operates on plans, and after every command the *coder* turns the plan back into program text, deciding where to nest expressions and where to introduce variables, and preferring names the programmer has already used. Waters called this two-way translation the *representation shift*, and argued that it gave plans locality, additivity, and modularity that text lacks [31]. Because the user may also simply edit the text, KBEmacs must keep text and plan consistent in both directions.

4.3 Running KBEmacs (headless)

KBEmacs depends heavily on the Lisp Machine environment of its time, including Flavors (the Lisp Machine's object system), the Zmacs editor, Lisp Machine processes, Waters's own LetS loop notation, and Zetalisp itself, which is similar to, but not exactly the same as, modern Common Lisp. Emulators survive for the MIT CADR Lisp Machine (usim) and for Symbolics Open Genera, but no usable environment compatible with this KBEmacs release (Symbolics Release 4.5) was available to us. Both include Flavors, but Open Genera, the closer of the two, is not freely available, and it corresponds to a Genera release about a decade later than the one KBEmacs ran on. The reanimation targeted the *headless* core (the database, analyzer, cliché library, command language, knowledge-based editor, and coder), leaving out the editor, windows, actors, and the Ada front end [20].

The 1985 source, with correction of minor typos and transcription errors, is loaded into SBCL (`https://www.sbcl.org`), a modern Common Lisp, through a Zetalisp compatibility layer that Claude assembled and debugged. The largest single piece of that layer is g000001's Common Lisp port of Spice Lisp Flavors [5].[4]

The rest of the layer supplies the parts of Zetalisp and the Lisp Machine that Common Lisp lacks or does differently. A small test harness was also added to replay the self-test sequence from the 1984 code and compare the generated code with the expected code.

The SQRT sequence (Figure 3) builds a square-root routine from three clichés in seven commands, and then builds a variant with an extra parameter in seven more. After the first command that names a cliché, KBEmacs holds the cliché's structure with its roles still open as holes; each further command fills a role with code or with another cliché, whose own unfilled roles appear as new holes. After seven commands the plan has 32 segments and the program is complete.

[4]Coincidentally, Spice Lisp (which was built at CMU, where Simon and Newell taught) became CMUCL, and SBCL is a fork of CMUCL.

```
>Reinitialize database.
>Define a program MYSQRT with a parameter NUM.
>Implement it as a successive-approximation.
>Implement the initial-approximation as [1.] .
>Implement the improvement-step as an average of [APPROXIMATION]
   and [(// NUM APPROXIMATION)] .
>Implement the criterion as an equality-within-epsilon of
   [(* APPROXIMATION APPROXIMATION)] and [NUM] .
>Implement the epsilon as [0.0001] .

(DEFUN MYSQRT (NUM)
  (LET* ((APPROXIMATION 1))
    (LOOP DO
      (IF (< (ABS (- (* APPROXIMATION APPROXIMATION) NUM)) 0.0001)
          (RETURN APPROXIMATION))
      (SETQ APPROXIMATION (// (+ APPROXIMATION (// NUM APPROXIMATION)) 2.0)))))

The program MYSQRT is a successive-approximation. The initial-approximation is 1.
The criterion is an equality-within-epsilon ...
```

Figure 3: KBEmacs, running in 2026, replaying the first half of the SQRT sequence from its December 14, 1984 self-test. The generated code is `equal` to the code recorded in 1984. The final lines are KBEmacs's own description of the program, generated from its plan, as recorded in the 1984 self-test (abridged).

All fourteen commands run, and both generated programs are `equal` (in the lisp sense) to the 1984 generated code. The generated code also works: `(mysqrt 2.0)` returns 1.4142157.

5 Then and Now: Two Historical Systems and Today's AI Coding Assistants

Neither the Heuristic Compiler nor KBEmacs is an ancestor of today's AI coding assistants in any direct technical sense. Large language models are trained, not programmed with explicit operators or clichés, and they operate on program text rather than on description lists or plans. Nonetheless the questions these two systems were built to explore are recognizably the questions that shape AI-assisted programming today. The comparisons that follow are conceptual continuities, not architectural equivalences.

How is a programming task specified? Simon's two compilers differ precisely in how the task is described. A state description says what the machine looks like before and after, which is close to what we would now call a specification by examples or by pre- and post-conditions, and close to the role unit tests play in directing a modern assistant. A functional description is an imperative phrase, which is close to a natural-language prompt. Simon drew a conclusion that remains apt: programmers use both, but "at the higher levels ... a language of functions appears generally the more natural and powerful," and "the mappings from functional descriptions to process language are simpler and more direct than the mappings from state descriptions" [25, p. 506]. KBEmacs chose the functional side and gave it a vocabulary of named algorithms. Today's assistants accept both kinds of description, and much of the craft of using them lies in combining a functional request with a test-like check.

Where does the knowledge come from? Both historical systems are knowledge-starved by modern standards. The Heuristic Compiler's state description compiler knows exactly two

operators; its functional description compiler can compile a routine only if a routine with the same process has already been compiled. KBEmacs knew "a few dozen" clichés; Waters estimated that a practical tool would need "many hundreds" [31, p. 55]. Both systems identified reuse of known solutions as the heart of the matter: Simon's functional compiler works by finding an analogous routine and adapting it, and the whole Programmer's Apprentice rests on the claim that programs are built from background knowledge. What neither could solve was acquiring that knowledge at scale. Large language models can be loosely described as one answer to precisely this bottleneck: in effect, an implicit cliché library absorbed from an enormous body of code. This is a metaphor, not a claim about what such models actually represent, and it comes at the price of having no explicit, inspectable representation of what a cliché is or what its roles are.

Who does what? Simon's Heuristic Compiler was not designed as a practical programming assistant. It was a computational experiment intended to illuminate the processes involved in programming. KBEmacs, by contrast, was deliberately conceived as an assistant, leaving design decisions to the programmer while taking over routine details. Waters's argument for the apprentice was practical: full automation was not near, and a semiexpert assistant could be useful at once. Modern tools have traversed the same axis in a few years, from completing a line, to writing a function on request, to agents that plan, edit, and test across a code base with varying degrees of supervision [27, 7]. The question of where the programmer's decisions end and the machine's begin, central to the Programmer's Apprentice, is again a live design question.

How does the system know it is making progress? The Heuristic Compiler's executive is a loop: apply a compiler, test whether the routine has moved to a better kind of definition, and stop if it has not. Its state description compiler implements difference reduction in the strict GPS sense. Modern coding agents commonly run a loop of a similar shape [27]: make a change, run the tests or the compiler, examine the difference between what happened and what was wanted, and try again. The difference detector is now an executed test suite rather than a hand-coded comparison of states, and the operator is a learned model rather than a table of connections, but the architecture of means-end iteration is recognizable. The resemblance goes beyond the loop. A coding agent of this kind is not a purely neural system: the language model chooses what to do next and interprets the results, but the processes of compiling, executing, comparing outputs, tracking changes, and so on, are done by conventional symbolic programs. In the work reported here these were a Common Lisp compiler, an IPL-V interpreter, and the 1984 self-test's own comparison of generated with expected code. Although the heuristics have moved from a hand-written table of connections into a learned model, the operators and the difference tests are still ordinary programs. KBEmacs, for its part, came with a self-test in which the expected output was code, which is exactly what made it possible to check its reanimation.

Representation and explanation. The two historical systems never treated code merely as text. The Heuristic Compiler builds IPL instructions out of description lists and prints them only at the end; KBEmacs reasons about plans and regenerates code after every command, taking pains to keep code and plan consistent when the user edits the code directly. That explicit representation is what allowed KBEmacs to describe a program in terms of its design ("The program MYSQRT is a successive-approximation ..."). Today's assistants operate on text and also explain code fluently, but their explanations are generated from text, not read off an explicit representation of the design. The trade-off between the robustness of working with text, which KBEmacs found painfully expensive to analyze—Waters reports that "well over half of all the effort expended on KBEmacs went into implementing an analyzer and coder" [31, p. 55]—and the inspectability of working with plans, has not gone away.

6 Next Steps

For the Heuristic Compiler, the largest piece of unexplored territory is the English front end: the scanner, dictionary, and parser, and the routine that converts a parse into a functional description. Simon credited H. S. Kelly with "linguistic components for the compiler which are omitted from this report" [25, p. 506]; these routines may be them. Running them requires input sentences, which are not in the listing.

For KBEmacs, the compatibility layer was debugged against the one self-test sequence whose expected output was known, so a skeptic may reasonably ask whether it is tuned to SQRT. The next step is therefore to run the other five sequences of the 1984 self-test without changing the layer after seeing their results, aiming to reproduce both the three successes and the three failures, and to build a new KBEmacs program that is not among the self-tests. After that comes the Ada front end, which would allow Waters's `UNIT-REPAIR-REPORT` demonstration to be reproduced.

7 Conclusion

These two reanimations illuminate a history of AI and programming that is not simply a progression from primitive to increasingly sophisticated coding assistants. Simon's Heuristic Compiler investigated the nature of programming as problem solving. KBEmacs applied AI to assist human programmers by combining explicitly represented programming knowledge. And today's AI coding agents take on more and more of the work of programming themselves. These systems are not stages in a single technical lineage. Rather, they mark three different relationships between machines and programming: using methods now central to AI[5] to understand programming, using AI to assist human programmers, and using AI to do more and more of the programming itself. By recovering and executing the original systems, executable archaeology allows us to examine these relationships through the detailed study of the computational mechanisms themselves.

There is also something these systems had that their modern successors lack. Like most early AI, both were in part attempts to understand how people do complex things. Simon's Heuristic Compiler treated programming as means-end problem solving: finding differences between what one has and what one wants, and applying operators that reduce them. The Programmer's Apprentice offered a different, and perhaps complementary, account: that expert programmers rarely work from first principles, but recognize and combine familiar clichés, each with roles to be filled. Each was a theory of programming that could be run and tested—and now can be run and tested again. Today's AI coding assistants program far better than either system, but they embody no comparable theory of how programming is done; whatever they know about clichés or differences is implicit in their weights, not stated where it can be examined. Executable archaeology recovers not only old programs, but the theories of programming that they made explicit.

8 Code Availability

The transcriptions, interpreters, compatibility layer, and Hacker's Guides to both programs are open source, at `https://github.com/jeffshrager/ExecutableArchaeology` and `https://github.com/jeffshrager/kbemacs` [19, 20]. Either the original source scans are located there as well, or links are provided to these scans.

[5] Newell and Simon themselves long preferred the term "complex information processing" to "artificial intelligence."

Author Contributions and Use of AI

J. Shrager conceived and directed the project, searched the Simon and Newell papers in the Carnegie Mellon University Archives with M. Golec, wrote the IPL-V interpreter on which the Heuristic Compiler runs, checked the results against the historical record, and edited the paper. K. M. Pitman, a member of the original KBEmacs team, provided the scans of the January 1985 KBEmacs listings and guidance on the Zetalisp compatibility layer required to run it. M. Golec worked with Shrager over an extended period in the Simon and Newell archives, and selected and scanned the Heuristic Compiler listing. All human authors read the paper and provided edits.

Most of the programming work described here was carried out by AI systems, under the authors' direction. Most of the transcription of the listings from the page images was done by ChatGPT (OpenAI) [3]. Claude (Anthropic), running as Claude Code [4] with the Claude Opus 5.5 model, corrected the transcriptions by running the code and checking the scans; adapted and extended Shrager's IPL-V interpreter; adapted g000001's Common Lisp ports of Spice Lisp Flavors and LetS [5], developed the Zetalisp compatibility layer for KBEmacs, and ran and debugged both programs until they reproduced their historical output. Claude also drafted this paper. The draft was then carefully checked by ChatGPT and by the authors, who verified its claims against the listings, the program runs, and the historical sources, and who take full responsibility for the final text.

These two reanimations required about 1,500 calls to the Claude Opus 5.5 model, about 1,600 tool invocations, and about six hours of active machine time (not counting ChatGPT's extraction of the KBEmacs listings from the scans).

Acknowledgments

We thank Emily Davis of Carnegie Mellon University, who supported Mia Golec's search of the Herbert A. Simon Papers in Carnegie Mellon University's Archives, resulting in locating the Heuristic Coder listing. We thank g000001 for the Common Lisp port of Spice Lisp Flavors and LetS (https://github.com/g000001/spice-flavors, https://github.com/g000001/LetS) [5]. RAND Memorandum RM-3588-PR ([26]) was obtained from bitsavers (http://bitsavers.org/pdf/rand/ipl/). The Heuristic Coder listing is copyright CMU archives, and KBEmacs is copyright MIT. These are reproduced under the notice in those listings.

Finally, we acknowledge Herb Simon, H. S. Kelly, Richard Waters, Charles Rich, and the other members of the Programmer's Apprentice project, on whose work this paper rests.

References

[1] J. W. Backus et al., "The FORTRAN automatic coding system," in *Proc. Western Joint Comput. Conf.*, 1957, pp. 188–198.

[2] R. Balzer, "A 15 year perspective on automatic programming," *IEEE Trans. Softw. Eng.*, vol. SE-11, no. 11, pp. 1257–1268, Nov. 1985.

[3] OpenAI, "ChatGPT," 2026. [Online]. Available: https://chatgpt.com

[4] Anthropic, "Claude Code," 2026. [Online]. Available: https://www.anthropic.com/claude-code

[5] g000001, "spice-flavors" and "LetS," GitHub. Accessed: Oct. 10, 2026. [Online]. Available: https://github.com/g000001/spice-flavors, https://github.com/g000001/LetS

[6] C. Green, "Application of theorem proving to problem solving," in *Proc. 1st Int. Joint Conf. Artif. Intell.*, 1969, pp. 219–240.

[7] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan, "SWE-bench: Can language models resolve real-world GitHub issues?" in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2024.

[8] R. Lane, A. Hay, A. Schwarz, D. M. Berry, and J. Shrager, "ELIZA reanimated: Restoring the mother of all chatbots to one of the world's first time-sharing systems," *IEEE Ann. Hist. Comput.*, vol. 47, no. 2, pp. 68–76, Apr.-Jun. 2025.

[9] Z. Manna and R. J. Waldinger, "Toward automatic program synthesis," *Commun. ACM*, vol. 14, no. 3, pp. 151–165, Mar. 1971.

[10] D. Moews and J. Shrager, "Executable archaeology: Reanimating the Logic Theorist," *IEEE Ann. Hist. Comput.*, submitted, 2026.

[11] A. Newell, J. C. Shaw, and H. A. Simon, "Report on a general problem-solving program," in *Proc. Int. Conf. Inform. Process.*, Paris: UNESCO, 1960, pp. 256–264.

[12] A. Newell, J. C. Shaw, and H. A. Simon, "Empirical explorations of the logic theory machine: A case study in heuristic," in *Proc. Western Joint Comput. Conf.*, 1957, pp. 218–230.

[13] K. M. Pitman, "netsettlement: kbe," GitHub (scans of *Knowledge-Based Editor*, four-volume listing, Jan. 1985). Accessed: Oct. 13, 2026. [Online]. Available: https://github.com/netsettler/netsettlement/tree/main/kbe

[14] C. Rich and H. E. Shrobe, "Initial report on a LISP programmer's apprentice," MIT AI Lab., Cambridge, MA, USA, Tech. Rep. AI-TR-354, Dec. 1976. (Also *IEEE Trans. Softw. Eng.*, vol. SE-4, no. 6, pp. 456–467, Nov. 1978.)

[15] C. Rich, "A formal representation for plans in the Programmer's Apprentice," in *Proc. 7th Int. Joint Conf. Artif. Intell. (IJCAI)*, Vancouver, BC, Canada, 1981, pp. 1044–1052.

[16] C. Rich and R. C. Waters, "The Programmer's Apprentice project: A research overview," MIT AI Lab., Cambridge, MA, USA, AI Memo 1004, Nov. 1987.

[17] C. Rich and R. C. Waters, "Automatic programming: Myths and prospects," *Computer*, vol. 21, no. 8, pp. 40–51, Aug. 1988.

[18] C. Rich and R. C. Waters, *The Programmer's Apprentice*. New York, NY, USA: ACM Press, and Reading, MA, USA: Addison-Wesley, 1990.

[19] J. Shrager, "Executable Archaeology," GitHub (HeuristicCoder, IPL-V, and LogicTheorist directories). Accessed: Oct. 16, 2026. [Online]. Available: https://github.com/jeffshrager/ExecutableArchaeology

[20] J. Shrager, "kbemacs: MIT's KBEmacs from the January 1985 listings, running under SBCL," GitHub. Accessed: Oct. 16, 2026. [Online]. Available: https://github.com/jeffshrager/kbemacs

[21] J. Shrager, "Executable archaeology: Reanimating the Logic Theorist from its IPL-V source," Mar. 13, 2026, *arXiv:2603.13514v1*. [Online]. Available: https://arxiv.org/abs/2603.13514

[22] J. Shrager. "An IPL-V interpreter in Common Lisp that runs the original Logic Theorist," GitHub (archived; superseded by [19]). [Online]. Available: https://github.com/jeffshrager/IPL-V

[23] H. A. Simon, "Experiments with a heuristic compiler," RAND Corp., Santa Monica, CA, USA, Paper P-2349, Jun. 30, 1961.

[24] H. A. Simon, "Heuristic Coder 7/16/61," IPL-V program listing, Herbert A. Simon Papers, Box 14, Folder FF965, Carnegie Mellon Univ. Archives, Pittsburgh, PA, USA.

[25] H. A. Simon, "Experiments with a heuristic compiler," *J. ACM*, vol. 10, no. 4, pp. 493–506, Oct. 1963.

[26] H. A. Simon, "The heuristic compiler," RAND Corp., Santa Monica, CA, USA, Memorandum RM-3588-PR, May 1963. [Online]. Available: http://bitsavers.org/pdf/rand/ipl/RM-3588-PR_The_Heuristic_Compiler_May63.pdf

[27] J. Yang, C. E. Jimenez, A. Wettig, K. Lieret, S. Yao, K. Narasimhan, and O. Press, "SWE-agent: Agent-computer interfaces enable automated software engineering," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

[28] R. C. Waters, "Automatic analysis of the logical structure of programs," Ph.D. dissertation, MIT, Cambridge, MA, USA, 1978. (MIT AI Lab. Tech. Rep. AI-TR-492.)

[29] R. C. Waters, "KBEmacs: A step toward the Programmer's Apprentice," MIT AI Lab., Cambridge, MA, USA, Tech. Rep. AI-TR-753, May 1985.

[30] R. C. Waters, "The Programmer's Apprentice: A session with KBEmacs," *IEEE Trans. Softw. Eng.*, vol. SE-11, no. 11, pp. 1296–1320, Nov. 1985.

[31] R. C. Waters, "KBEmacs: Where's the AI?" *AI Mag.*, vol. 7, no. 1, pp. 47–56, Spring 1986.

About the Authors

Jeff Shrager is a cognitive scientist and computer scientist who is currently Chief Scientist of Bennu Climate, Inc., a methane removal company. Jeff was a student of Herb Simon's at CMU in the 1980s, and taught in the SymSys program at Stanford for almost 25 years (as an adjunct). His primary intellectual project is CogLean, a Lean-based library of formal components that can be used to prove claims about cognitive models. In his spare time Jeff archive dives to find early AIs, and then works with modern AIs to reanimate them.
Kent M. Pitman {Kent M. Pitman to provide}
Mia Golec {Mia Golec to provide}